



IILM UNIVERSITY

School of Computer Science and Engineering

C/C++ PROGRAMMING FUNDAMENTALS AND PRACTICAL IMPLEMENTATION

SUMMER INTERNSHIP PRESENTATION

Anurodh Singh

B.Tech CSE • IILM University

Intern ID CTI/A1/C337566 • 11 Apr - 11 May 2026

Cognifyz IT Solutions Pvt. Ltd. | C/C++ Programming Intern
11 April 2026 - 11 May 2026

Faculty Coordinator: _____

Internship at a Glance

A one-screen view of the internship and domain context

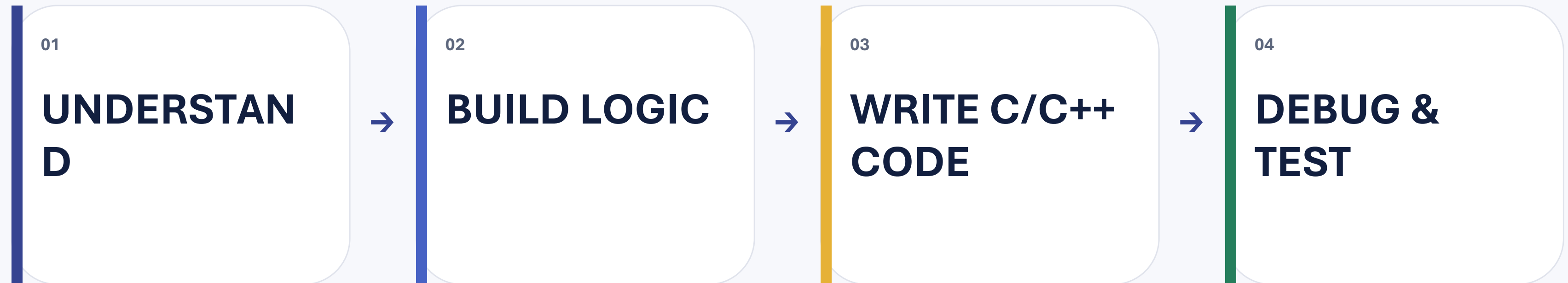
ORGANIZATION Cognifyz IT Solutions Pvt. Ltd.	ROLE C/C++ Programming Intern	DURATION 11 Apr - 11 May 2026	INTERN ID CTI/A1/C337566
PROJECT / DOMAIN C/C++ Programming Fundamentals and Practical Implementation			
INTERNSHIP CONTEXT The certificate confirms the role of C/C++ Programming Intern. This presentation organizes relevant C/C++ concepts such as programming logic, functions, OOP, debugging and problem solving around that verified domain.			

Based on the supplied internship completion certificate.

Certificate details are reproduced exactly; technical sections are aligned to the C/C++ domain.

My Role as a C/C++ Programming Intern

The internship domain connects problem understanding with clean implementation



Internship role: C/C++ Programming Intern

The certificate verifies the programming domain and highlights dedication, communication, attention to detail and a strong desire to learn.

Project Overview

From programming concepts to structured C/C++ solutions



UNDERSTAND → DESIGN → IMPLEMENT → COMPILE → TEST

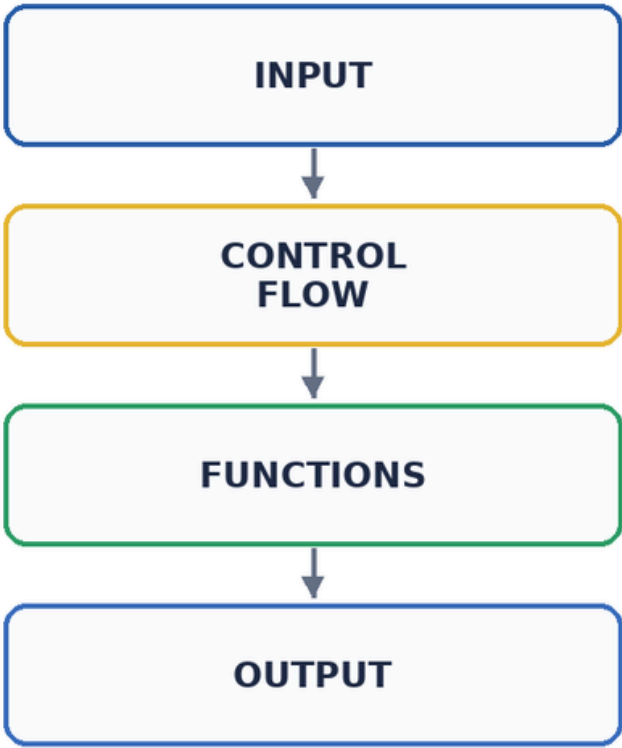
Core technical dimensions represented in the report: syntax, control flow, functions, arrays, strings, pointers, structures, OOP, standard library concepts, file handling and debugging.

C Fundamentals & Program Structure

First step: understand how a C program is organized and executed

C PROGRAM STRUCTURE

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```



Syntax -> logic -> execution

FOUNDATION

Syntax

CONTROL FLOW

if / loops

MODULARITY

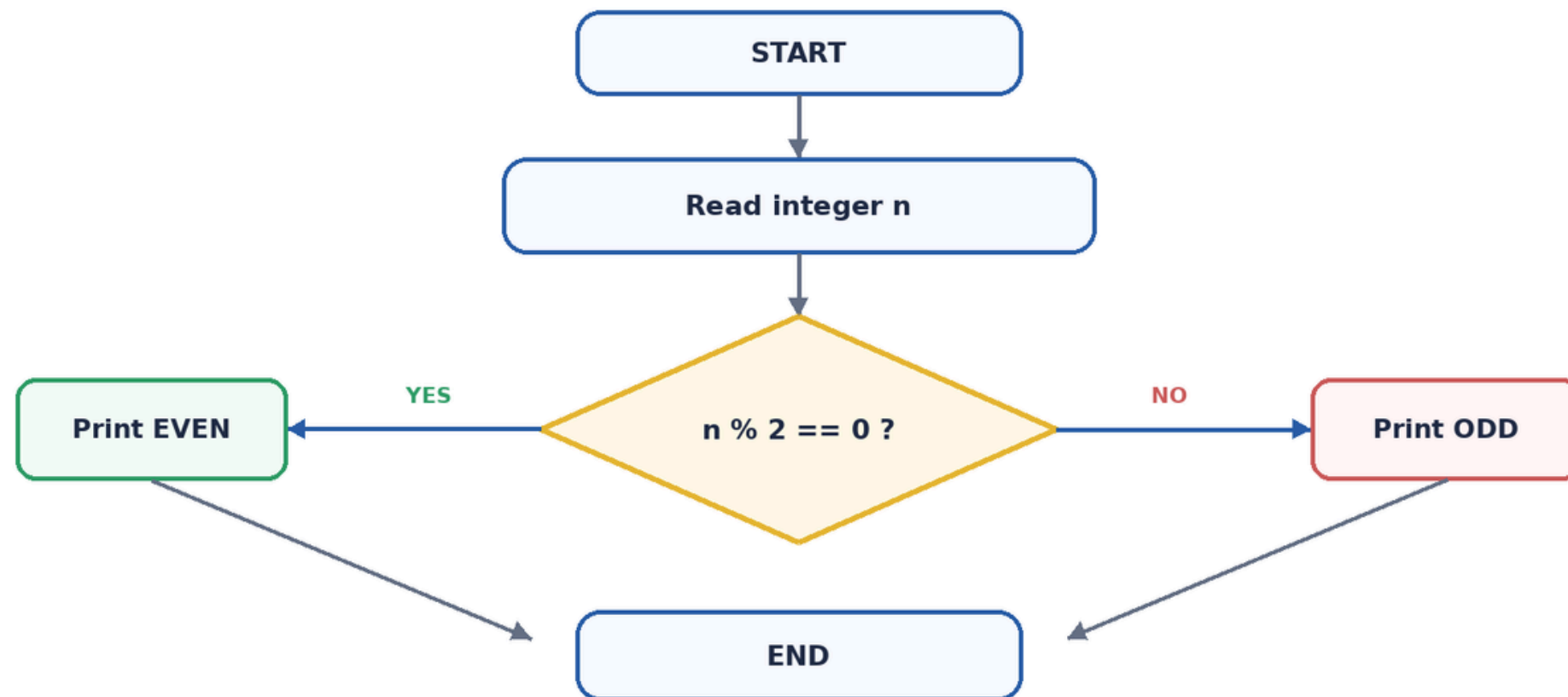
Functions

Core flow: input -> condition/loop -> function -> output

Logic Building: The First Important Skill

Programming solutions begin with a correct step-by-step algorithm

LOGIC BUILDING WITH CONTROL FLOW



CONDITIONS

if / else

decision making

LOOPS

for / while

repetition

FUNCTIONS

Reusable logic

INTERPRETATION

Before writing code, the problem should be converted into clear conditions, loops and reusable steps.

Functions & Modular Programming

Breaking a solution into smaller reusable components

C: FUNCTION-BASED MODULARITY

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int result = add(10, 20);  
    printf("%d", result);  
    return 0;  
}
```

C function example

C++: CLASS AND OBJECT

```
class Student {  
private:  
    int marks;  
public:  
    void setMarks(int m) {  
        marks = m;  
    }  
};  
Student s;  
s.setMarks(90);
```

C++ class-based example

Core C Concepts

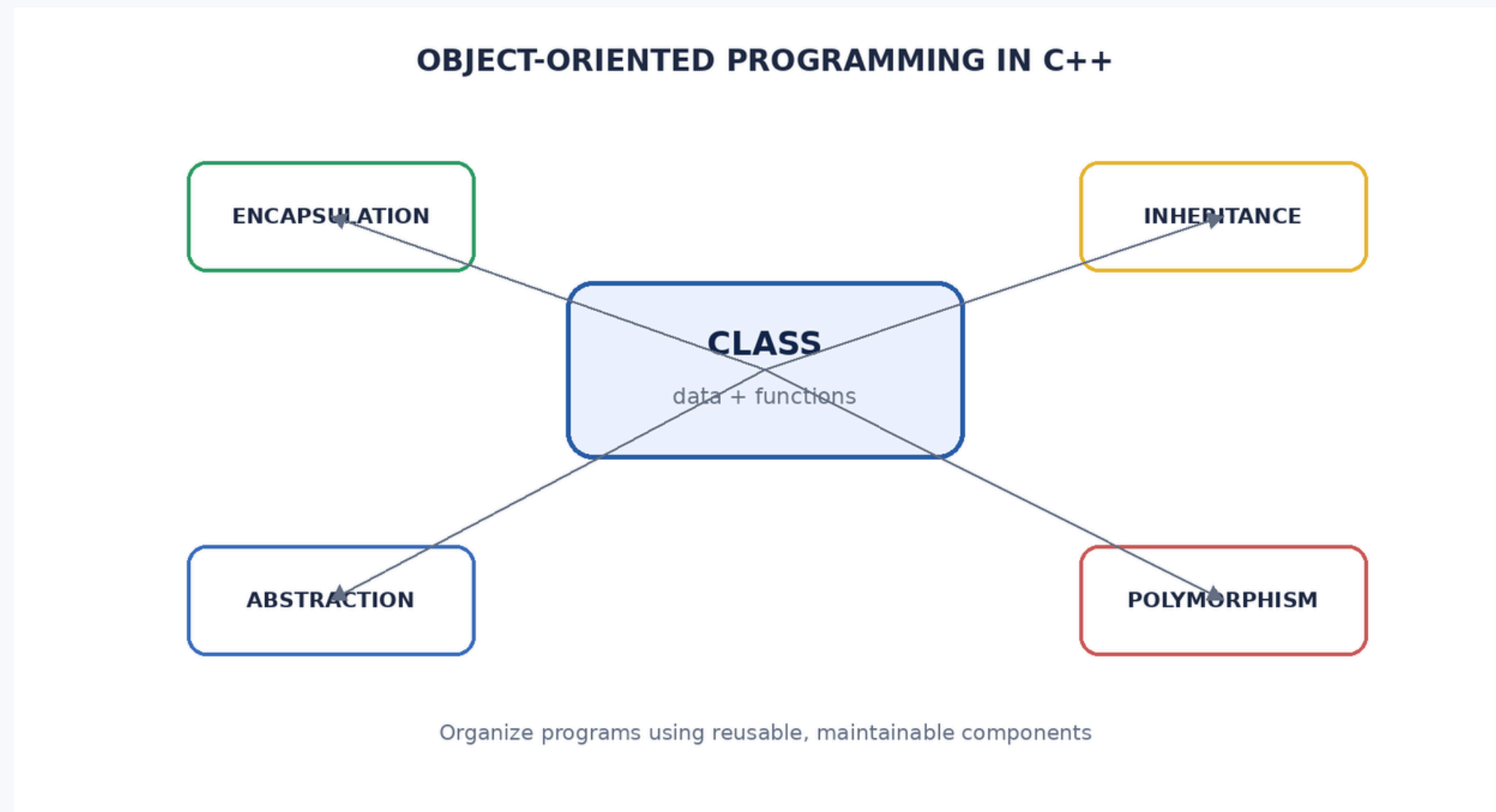
The foundation of C/C++ programming is built from a small set of reusable concepts

01	Variables & Control Flow		Core Syntax
02	Arrays & Strings		Data Handling
03	Functions		Modularity
04	Pointers & Structures		Memory / Records
05	File Handling		Persistent I/O

C fundamentals provide the base for structured problem solving and later C++ concepts.

Object-Oriented Programming in C++

C++ introduces reusable program design through classes and objects



CLASS & OBJECT

Program unit

data + functions

ENCAPSULATION

**Controlled
access**

private / public

INHERITANCE

Code reuse

derived classes

POLYMORPHISM

**Flexible
behavior**

overloading / overriding

OOP helps organize larger programs into reusable, maintainable components.

Standard Library & C++ Tools

Ready-made library components reduce repeated implementation effort

C++ CONTAINER

vector

Dynamic sequence

TEXT HANDLING

string

Character data

SEARCH & SORT

algorithms

Reusable operations

WHY THIS MATTERS

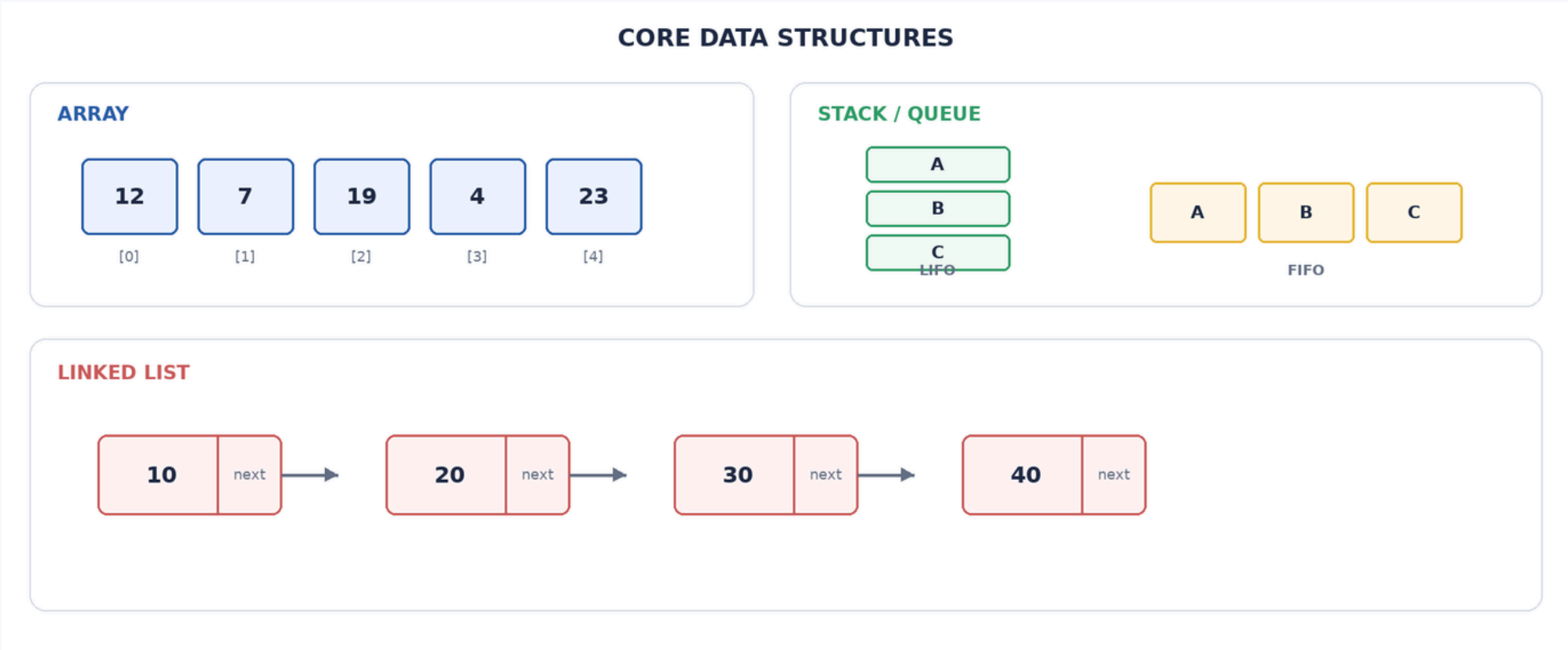
Standard library containers and algorithms can make C++ programs shorter, clearer and easier to maintain.

ANALYTICAL TAKEAWAY

Choose the simplest suitable container or algorithm for the problem instead of rebuilding common operations from scratch.

Data Structures & Algorithms

Core structures support efficient storage, traversal and problem solving



CORE STRUCTURES

Array / Stack

Queue / List

SEARCHING

Linear / Binary

Search methods

SORTING

Basic sorts

Order data

The diagram summarizes common structures that are frequently implemented or used in C/C++ problem solving.

Debugging: Why Correct Logic Matters

Programming quality depends on identifying errors systematically

SYNTAX ERRORS

Compiler-detected Wrong output

missing token / type issue

LOGIC ERRORS

incorrect condition / boundary

KEY INTERPRETATION

A program can compile successfully and still produce incorrect output.

Dry runs, sample inputs and boundary cases help separate syntax, runtime and logic problems.

File Handling & Record Management

Files allow a console program to store and retrieve data beyond one execution

FILE HANDLING WORKFLOW



C++ STREAM EXAMPLE

```
ofstream file("records.txt");  
file << name << " " << marks;  
file.close();
```

READ

**Input
from file**

WRITE

**Save
records**

VALIDATION

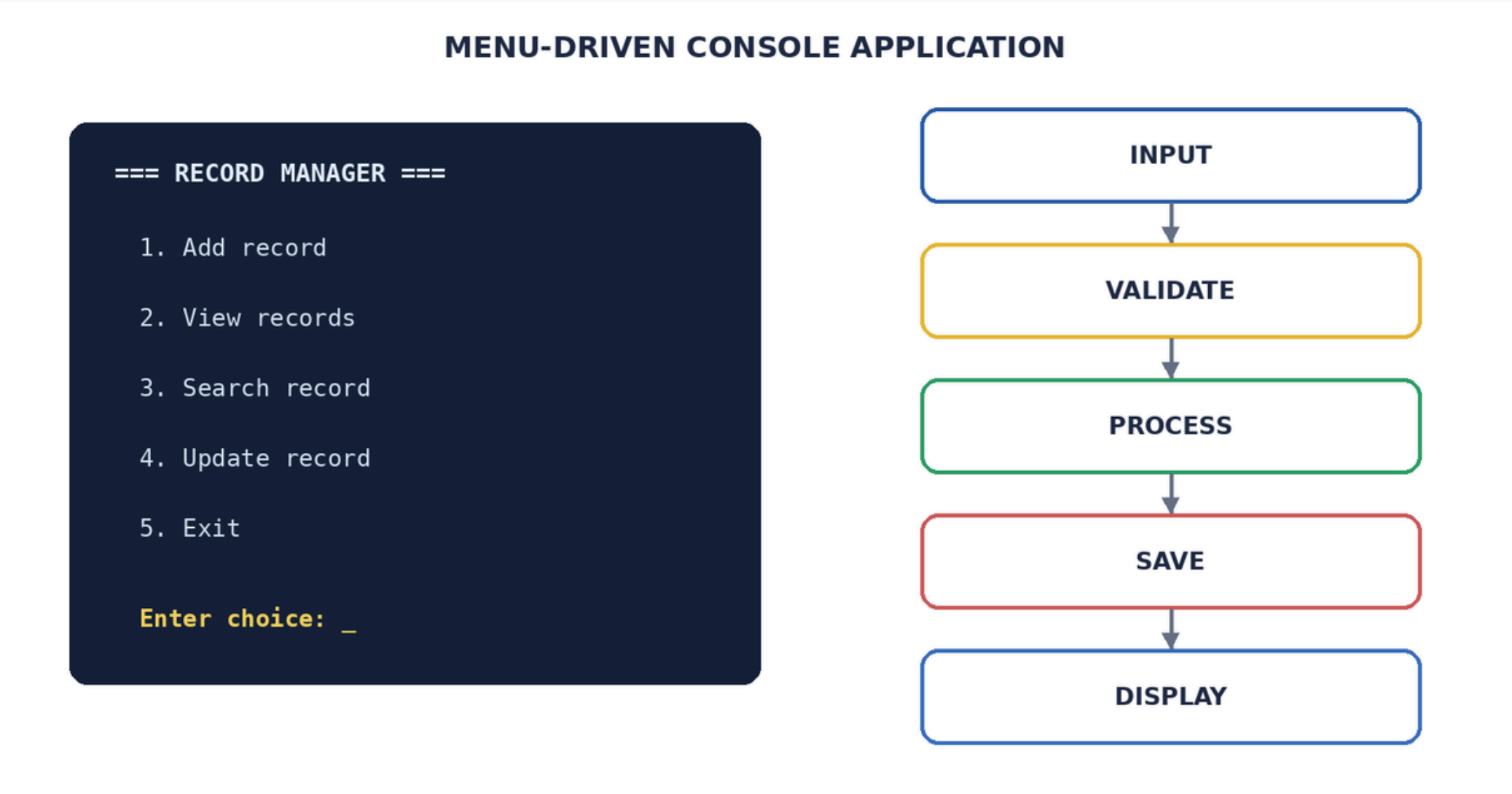
Check open / input

safe access

**File streams connect program logic with
persistent records.**

Console Application Workflow

A menu-driven program combines input, processing, validation and output



INPUT

User choice

menu selection

VALIDATE

Check values

invalid input

PROCESS

Apply logic

functions / classes

SAVE

File / memory

persistent record

A structured menu keeps the program flow understandable and easy to test.

Testing & Output Verification

Compilation, dry runs and test cases improve reliability

COMPILE - TEST - DEBUG CYCLE



Test Case	Input	Expected	Check
1	5	Prime	PASS
2	10	Not Prime	PASS
3	1	Not Prime	PASS

TEST CASES

Normal + edge inputs

DEBUGGING

Correct logic

FINAL OUTPUT

Verified result

QUALITY CHECK

The goal is not only to run the program, but to verify that it behaves correctly for expected and boundary inputs.

Code Quality Practices

Converting raw logic into readable and maintainable programs

Variables

Descriptive Names →

Functions

Modular Design →

Input

Validation →

Comments

Documentation →

STL / Library

Reuse →

Style

Consistent Format →

Good code should be understandable, testable and easy to modify.

Professional Qualities Highlighted in the Certificate

The certificate records both learning attitude and communication-related strengths

QUALITIES HIGHLIGHTED IN THE CERTIFICATE

Dedication	Sincerity
Strong desire to learn	Coordination skills
Effective communication	Attention to detail
Enthusiasm	Commitment to learning

CERTIFICATE HIGHLIGHT

Strong desire to learn

explicitly noted

PROFESSIONAL QUALITIES

- Dedication and sincerity
- Coordination skills
- Effective communication
- Attention to detail

Interpretation: these qualities complement technical programming practice and support reliable teamwork.

End-to-End Methodology

A structured workflow for solving C/C++ programming tasks



Problem Statement → **Algorithm / Logic** → **C/C++ Program** → **Verified Output**

A good programming workflow moves from understanding to implementation, verification and explanation.

Key Internship Outcomes

Certificate-verified qualities combined with role-aligned technical learning

PROGRAMMING DOMAIN

C/C++ role confirmed by certificate

COMMUNICATION

Effective communication abilities

DETAIL

Attention to detail highlighted

LEARNING ATTITUDE

Strong desire to learn

COORDINATION

Exceptional coordination skills

COMMITMENT

Willingness to acquire knowledge and skills

The internship certificate confirms successful participation in the C/C++ Programming domain and recognizes dedication, sincerity, communication, coordination and commitment to learning.

Learning Outcomes

Technical concepts organized around the verified C/C++ programming domain

C

Program structure and control flow

C++

Classes and object-oriented concepts

Functions

Modular problem solving

Data Structures

Arrays, stack, queue and lists

Debugging

Compile, test and refine

Communication

Explain logic and outputs clearly

From problem statement to verified program

The internship domain supports both programming practice and the professional qualities recorded in the certificate.

Internship Completion

Official completion certificate supplied by the user



Certificate dated 11 May 2026.

ORGANIZATION

Cognifyz IT Solutions Pvt. Ltd.

C/C++ Programming Intern

INTERNSHIP DURATION

11 Apr - 11 May 2026

Intern ID: CTI/A1/C337566

The supplied certificate confirms Anurodh Singh successfully completed the internship as a C/C++ Programming Intern.



IILM UNIVERSITY

THANK YOU

C/C++ PROGRAMMING FUNDAMENTALS AND PRACTICAL IMPLEMENTATION

Anurodh Singh • Intern ID CTI/A1/C337566

B.Tech | IILM University | 11 Apr - 11 May 2026

Cognifyz IT Solutions Pvt. Ltd. • C/C++ Programming Intern